

Lezione 12 – DevSecOps: sicurezza nella pipeline CI/CD

Durata indicativa: 1 ora

Obiettivi

- ## La pipeline

```
commit / push
```

```
|  
+--- Snynk: SCA + SAST (gate HIGH/CRITICAL)  
+--- Trivy: image CVE scan (gate HIGH/CRITICAL) --> DAST  
+--- Trivy: secrets + IaC (solo report)  
  
                                     |  
                                build e start del container  
                                   |  
                             health check /api/health  
                                 |  
                   OWASP ZAP baseline (DAST gate)
```

Job	Controllo	Comportamento
snyk	Snyk Open Source e Snyk Code	fallisce con finding HIGH/CRITICAL
trivy-image	CVE dell'immagine Docker	fallisce con CVE HIGH/CRITICAL; pubblica SARIF
trivy-repository	segreti e IaC	report non bloccante, adatto alla demo
dast-zap	OWASP ZAP contro il container	avvia il container e fallisce sugli alert ZAP

La scansione ZAP è una **baseline scan**: esplora l'app e applica controlli passivi. È sicura per questa demo e non è un penetration test completo.

Prerequisiti GitHub

1. In **Settings** → **Secrets and variables** → **Actions**, crea il secret SNYK_TOKEN.
2. Fai commit e push; apri la scheda **Actions** del repository per proiettare l'esecuzione in aula.

```
git add .
git commit -m "Add DevSecOps security pipeline"
git push origin main
```

SNYK_TOKEN è l'unico segreto richiesto. Per pull request da fork GitHub non espone i secrets: in quel caso i job Snyk non possono autenticarsi, mentre Trivy e ZAP non richiedono credenziali.

Prova locale

```
docker build -t gestionale-app:local .
docker run -d --rm --name gestionale-demo -p 8080:8080 gestionale-app:local
```

```
curl http://localhost:8080/api/health
curl http://localhost:8080/api/studenti
```

```
trivy image --severity CRITICAL,HIGH gestionale-app:local
trivy fs --scanners vuln,secret,config .
```

```
docker stop gestionale-demo
```

Per Snyk in locale:

```
npm install -g snyk
snyk auth
snyk test --file=requirements.txt --severity-threshold=high
snyk code test . --severity-threshold=high
```

Per una prova DAST manuale, con il container già attivo:

```
docker run --rm --network host -t ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t http://127.0.0.1:8080 -r zap-report.html
```

Cosa osservare in aula

- Aprire un run nella scheda **Actions** e mostrare i job paralleli.
- Aprire dast-zap: si vedono build, container, health check e report ZAP.
- Introdurre una dipendenza vulnerabile o una configurazione errata, ripetere push e discutere il gate che blocca la pipeline.
- I risultati Trivy SARIF sono disponibili in **Security** → **Code scanning** quando la funzionalità è abilitata nel repository.

Checkpoint

- ☐ SNYK_TOKEN configurato nel repository
- ☐ un push ha eseguito tutti e quattro i job
- ☐ il container è stato raggiunto da ZAP tramite /api/health
- ☐ almeno un risultato è stato letto e discusso

Riferimenti

- [Snyk Python GitHub Action](#)
- [Trivy Action](#)
- [OWASP ZAP baseline scan](#)
- [GitHub: upload SARIF](#)